

Simulador Orbital - "ORB-SYS // HUD"

24/10/2026

Leonardo Lopez Pineda
Miguel Ángel Moreno Vázquez
Fernando José Camba Méndez
CDMX

1. Resumen del Proyecto y Concepto Visual

ORB-SYS // HUD es una aplicación web interactiva de página única (SPA) diseñada para simular, visualizar y contrastar la mecánica orbital y gravitatoria en 2D. El usuario asume el rol de un operador frente a una consola de telemetría espacial, seleccionando una estrella de un catálogo estelar real (escalado) e inyectando cuerpos celestes en órbita para analizar su comportamiento dinámico y energético.

Dirección de Arte: Sci-Fi Analógico (Cassette Futurism)

En lugar de requerir modelos 3D pesados, el impacto visual se logra emulando la interfaz de navegación de una nave espacial retro-futurista (inspirada en consolas de telemetría de los años 70/80, osciloscopios y monitores CRT de fósforo):

- **Paleta de Colores:** Fondo en negro profundo espacial (#080A0F), rejillas de coordenadas tenues en verde fósforo o cian (#00FFCC al 15% de opacidad), y alertas o vectores en ámbar (#FFB000) y carmesí (#FF3366).

- **Tipografía:** Fuentes monoespaciadas técnicas (ej. *JetBrains Mono*, *Share Tech Mono* o *VT323*) para reforzar la lectura de datos tabulares en tiempo real.

- **Propuesta de Renderizado 2D en HTML5 Canvas:**
  - **Persistencia de Fósforo (Estelas Orbitales):**
    En lugar de ejecutar clearRect() en cada fotograma, se dibuja un rectángulo del color de fondo con baja opacidad (rgba(8, 10, 15, 0.12)). Esto genera una cola luminosa orgánica que dibuja la trayectoria circular o elíptica sin costo extra de memoria.
  - **Resplandor Estelar (Efecto Bloom 2D):**
    Uso de ctx.globalCompositeOperation = 'lighter' combinado con gradientes radiales (createRadialGradient) y ctx.shadowBlur para que el núcleo de la estrella brille intensamente según su clase espectral.
  - **Vectores Dinámicos en Pantalla:**
    Dibujo directo sobre el cuerpo orbital de flechas vectoriales para la **Velocidad Tangencial ($\vec{v}_t$)** y la **Fuerza Centrípeta / Gravitatoria ($\vec{F}_g$)** que escalan su longitud en tiempo real.

2. Matriz de Cumplimiento de Requisitos Académicos

Requisito Solicitado	Implementación en el Simulador
1. Fenómeno físico claramente definido	Mecánica orbital de dos cuerpos: interacción entre la Ley de Gravitación Universal, el **Movimiento Circular** Uniforme/Variado, **Vectores,** y la conservación de la energía mecánica.
2. Modificar al menos dos variables físicas	El usuario puede modificar: **1)** Tipo/Masa de la estrella central (M), **2)** Masa del cuerpo orbital (m), **3)** Radio o distancia inicial (r), **4)** Velocidad tangencial inicial (v_t).
3. Mostrar resultados numéricos y gráficos	**Numéricos:** Telemetría en vivo de fuerzas, aceleración, velocidades y energía. **Gráficos:** Trazado de la órbita con vectores en el Canvas + minigráfica de barras/líneas de Energía Cinética (K) vs. Potencial (U).
4. Ecuaciones y relaciones físicas correctas	Integración vectorial exacta de gravitación, descomposición de aceleración centrípeta y tangencial, y cálculo de energía mecánica total del sistema.
5. Comparación: Simulador vs. Cálculo Teórico	**Módulo de Validación Teórica:** Un panel dedicado que calcula analíticamente la *Velocidad Orbital Circular Ideal (* v_{teo}*)* y el *Período de Kepler (*T_{teo}*)* para el radio inicial, contrastándolos con el período medido en el simulador (T_{sim}) y mostrando el porcentaje de desviación o excentricidad.

3. Flujo de Navegación y Experiencia de Usuario (UX)

La aplicación funciona mediante una máquina de estados simple en una sola vista web, sin necesidad de recargar la página ni gestionar bases de datos:

1. **Estado 1: Mapa Estelar (Selección de Estrella):**
   - El usuario visualiza un mapa estelar interactivo con estrellas preconfiguradas según su clasificación espectral:
     - **Clase M (Enana Roja):** Baja masa, radio pequeño, brillo rojizo tenue.
     - **Clase G (Enana Amarilla - Tipo Sol):** Masa media (referencia base), brillo dorado.
     - **Clase O (Gigante Azul):** Masa extrema, pozo gravitatorio intenso, resplandor azul-blanco.
     - **Estrella de Neutrones (Opcional/Bonus):** Masa altísima concentrada en un radio mínimo para probar velocidades extremas.
   - Al elegir una estrella, sus parámetros ($M_{estrella}$, radio visual, color) quedan bloqueados como centro del sistema.

2. **Estado 2: Configuración e Inyección de Objetos Orbitales:**
   - En la vista del sistema, el usuario selecciona el punto de inserción (distancia radial r) y ajusta mediante controles deslizantes (*sliders*) o entrada numérica: la **masa del objeto (m)** y su **velocidad tangencial inicial (v_t)**.
   - Incluye un botón de *"Auto-Calibrar Velocidad Circular"* que asigna la velocidad teórica exacta para que el usuario pueda probar primero el movimiento circular perfecto y luego alterarlo manualmente.

3. **Estado 3: Simulación Activa y Telemetría:**
   - Controles de reproducción: *Play / Pause*, *Paso a Paso (Step)* y multiplicador de velocidad de tiempo (*Time Warp: 0.5x, 1x, 2x, 5x*).
   - El panel HUD muestra los datos del objeto seleccionado y la comparativa teórica en tiempo real.

4. **Estado 4: Reinicio o Cambio de Sistema:**
   - **Botón "Reiniciar Simulación":** Limpia los objetos en órbita y restablece el tiempo a $t = 0$ manteniendo la misma estrella.
   - **Botón "Regresar al Mapa Estelar":** Destruye la instancia actual y devuelve al usuario al Estado 1.

4. Modelo Físico y Matemático

Para evitar trabajar con constantes del tamaño de $10^{30} kg$ en pantalla, el motor utilizará **Unidades Astronómicas de Simulación (UAS)** o escalamiento proporcional donde G está normalizada, mostrando al usuario las equivalencias en unidades reales (Masas Solares, km/s, Julios).

4.1. Gravitación Universal y Fuerza Centrípeta

La fuerza de atracción gravitatoria que la estrella de masa M ejerce sobre el objeto de masa m a una distancia $r = \sqrt{\Delta x^2 + \Delta y^2}$ es:

$$F_g = G\frac{M \cdot m}{r^2}$$

En una órbita circular estable, esta fuerza gravitatoria actúa exactamente como la **Fuerza Centrípeta (F_c)** necesaria para curvar la trayectoria del objeto:

$$F_c = \frac{m \cdot v_t^2}{r}$$

4.2. Velocidad Tangencial y Cómo Modifica la Aceleración

Por la Segunda Ley de Newton, la magnitud de la aceleración total inducida por la gravedad es $a = \frac{F_g}{m} = \frac{G \cdot M}{r^2}$. Sin embargo, la relación entre la **velocidad tangencial (v_t)** que el usuario introduce y la distancia r determina la geometría del movimiento a través de la **aceleración centrípeta (a_c)** y la **aceleración tangencial (a_t)**:

$$a_c = \frac{v_t^2}{r}$$

- **Caso 1: Órbita Circular Perfecta ($v_t = \sqrt{\frac{G \cdot M}{r}}$):** La aceleración gravitatoria iguala exactamente a la aceleración centrípeta requerida ($a_g = a_c$). El vector velocidad es siempre perpendicular al vector fuerza (90°), por lo que la aceleración tangencial es nula ($a_t = 0$) y la rapidez se mantiene constante.
- **Caso 2: Órbita Elíptica o Caída ($v_t \neq \sqrt{\frac{G \cdot M}{r}}$):** Si el usuario reduce o aumenta v_t (sin superar la velocidad de escape), el objeto se acerca o aleja de la estrella. El ángulo

entre la velocidad y la gravedad deja de ser de 90°, apareciendo una componente de **aceleración tangencial ($a_t = \frac{\vec{a}\cdot\vec{v}}{\|\vec{v}\|}$)** que acelera al objeto cuando cae hacia el periastro y lo frena cuando sube hacia el apoastro.

- **Caso 3: Escape Hiperbólico ($v_t \geq \sqrt{\frac{2\cdot G\cdot M}{r}}$):** La energía cinética supera el pozo gravitatorio y el objeto escapa del sistema.

4.3. Energía Mecánica del Sistema

En cada fotograma se calcula el balance energético para graficar la transformación entre energía cinética y potencial:

$$K = \frac{1}{2}mv^2, U = -\frac{G\cdot M\cdot m}{r}, E_{total} = K + U$$

- En movimiento circular uniforme, K y U se mantienen como líneas rectas constantes.
- En órbitas elípticas, K sube cuando U baja, manteniendo E_{total} constante (demostración visual de la Conservación de la Energía).

4.4. Ecuaciones del Módulo de Comparación Teórica vs. Simulador

Al insertar un objeto a una distancia inicial r_0, el sistema calcula de forma analítica los valores teóricos de una órbita circular ideal y los compara con los datos medidos en el bucle de simulación:

Velocidad Circular Teórica vs. Velocidad Actual:

$$v_{teo} = \sqrt{\frac{G\cdot M}{r_0}}$$

Período Orbital Teórico (3ra Ley de Kepler) vs. Período Medido por Cruce de Ángulo ($2\pi rad$):

$$T_{teo} = 2\pi\sqrt{\frac{r_0^3}{G \cdot M}} vs. T_{sim} = t_{vuelta_completa}$$

Cálculo de Error / Desviación Porcentual:

$$\%\text{Error} = \left| \frac{\text{Valor}_{teo} - \text{Valor}_{sim}}{\text{Valor}_{teo}} \right| \times 100$$

5. Arquitectura de Software y Distribución del Código

Se propone un stack moderno, tipado y sin dependencias pesadas: **Vite + TypeScript + HTML5 Canvas + CSS nativo (Grid/Custom Properties)**. No requiere servidor backend ni base de datos.

```
src/
├── main.ts                  # Punto de entrada e inicialización del ciclo requestAnimationFrame
├── config/
│   └── StarPresets.ts       # Diccionario con constantes físicas de estrellas (Clases M, G, K, O)
├── physics/
│   ├── Vector2D.ts          # Operaciones vectoriales (suma, magnitud, producto punto, normalización)
│   ├── CelestialBody.ts     # Clase de entidad orbital (posición, velocidad, masa, historial de estela)
│   ├── PhysicsEngine.ts     # Cálculo de gravitación, energías, componentes centrípetas e integrador
│   └── TheoreticalCalc.ts   # Fórmulas analíticas cerradas y cálculo de porcentaje de error
├── render/
│   └── CanvasRenderer.ts    # Dibujo 2D: estelas con alpha decay, bloom estelar, rejilla HUD y vectores
└── ui/
    └── HUDController.ts     # Manejo de vistas (Mapa Estelar / Simulador), sliders y tablas de telemetría
```

Detalle de las Funciones Clave del Motor Físico (`PhysicsEngine.ts`)

Para evitar que los errores de redondeo destruyan la órbita con el paso del tiempo, se utiliza **Integración de Euler Semi-Implícita (Simpéctica)**, donde primero se actualiza la velocidad con la aceleración actual y luego la posición usando esa *nueva* velocidad:

(Typescript generado, no representa producto final, no está revisado)

PhysicsEngine.ts

```typescript
export interface TelemetryData {
  distance: number;
  speed: number;
  gravitationalForce: number;
  requiredCentripetalForce: number;
  centripetalAccel: number;
  tangentialAccel: number;
  kineticEnergy: number;
  potentialEnergy: number;
  totalEnergy: number;
  theoreticalSpeed: number;
  theoreticalPeriod: number;
  simulatedPeriod: number | null;
}

export class PhysicsEngine {
  // Constante gravitacional escalada para unidades del simulador
  public static readonly G = 1000;

  /**
   * Actualiza la física de un cuerpo orbital en un paso de tiempo dt
   */
  public static step(starMass: number, starPos: Vector2D, body: CelestialBody, dt: number): void {
    // 1. Vector dirección desde el cuerpo hacia la estrella
    const rVec = Vector2D.sub(starPos, body.pos);
    const r = rVec.magnitude();
    const rHat = rVec.normalize(); // Vector unitario radial

    // 2. Magnitud de la Fuerza de Gravitación Universal: F = G * M * m / r^2
    const forceMag = (this.G * starMass * body.mass) / (r * r);
    const forceVec = Vector2D.scale(rHat, forceMag);

    // 3. Aceleración gravitatoria total: a = F / m
    const accelVec = Vector2D.scale(forceVec, 1 / body.mass);

    // 4. Integrador Euler Semi-Implícito (conserva la energía orbital)
    body.vel = Vector2D.add(body.vel, Vector2D.scale(accelVec, dt));
    body.pos = Vector2D.add(body.pos, Vector2D.scale(body.vel, dt));

    // 5. Registro de ángulo barrido para medir el Período Orbital Simulado (T_sim)
    body.updateOrbitalPeriodTracker(starPos, dt);
  }
```

```
  /**
   * Calcula todas las variables físicas y la comparación teórica para el HUD
   */
  public static computeTelemetry(starMass: number, starPos: Vector2D, body: CelestialBody): TelemetryData {
    const rVec = Vector2D.sub(starPos, body.pos);
    const r = rVec.magnitude();
    const v = body.vel.magnitude();

    // Fuerzas
    const gravitationalForce = (this.G * starMass * body.mass) / (r * r);
    const requiredCentripetalForce = (body.mass * v * v) / r;

    // Descomposición de aceleración (Tangencial vs Centrípeta/Normal)
    const rHat = rVec.normalize();
    const vHat = body.vel.normalize();
    const accelVec = Vector2D.scale(rHat, (this.G * starMass) / (r * r));

    const tangentialAccel = Vector2D.dot(accelVec, vHat); // Proyección sobre la velocidad
    const centripetalAccel = (v * v) / r;

    // Energías del Sistema
    const kineticEnergy = 0.5 * body.mass * (v * v);
    const potentialEnergy = -(this.G * starMass * body.mass) / r;
    const totalEnergy = kineticEnergy + potentialEnergy;

    // Cálculos Teóricos (tomando el radio inicial de inserción r0 del cuerpo)
    const r0 = body.initialRadius;
    const theoreticalSpeed = Math.sqrt((this.G * starMass) / r0);
    const theoreticalPeriod = 2 * Math.PI * Math.sqrt(Math.pow(r0, 3) / (this.G * starMass));

    return {
      distance: r,
      speed: v,
      gravitationalForce,
      requiredCentripetalForce,
      centripetalAccel,
      tangentialAccel,
      kineticEnergy,
      potentialEnergy,
      totalEnergy,
      theoreticalSpeed,
      theoreticalPeriod,
      simulatedPeriod: body.lastCompletedPeriod
    };
  }
}
```

6. Plan de Desarrollo por Fases (Roadmap del Equipo)

Para organizar el trabajo entre los integrantes del equipo sin bloqueos, el desarrollo se divide en 4 fases modulares:

Fase 1: Núcleo Matemático y Motor Físico

- **Objetivo:** Programar las clases Vector2D, CelestialBody, PhysicsEngine y TheoreticalCalc completamente aisladas de la interfaz gráfica.
- **Entregable verificable:** Pruebas en consola donde un objeto colocado a una distancia r_0 con velocidad $v_{teo} = \sqrt{GM/r_0}$ mantiene su radio constante tras 1,000 iteraciones y devuelve un período simulado T_{sim} con menos de 0.5\% de diferencia respecto a T_{teo}.

Fase 2: Renderizador 2D y Efectos Sci-Fi en Canvas **(puede ser AI Generated)**

- **Objetivo:** Construir CanvasRenderer.ts e integrarlo con el bucle requestAnimationFrame a 60 FPS.
- **Entregable verificable:** Lienzo funcional que dibuja la cuadrícula espacial, renderiza la estrella central con su halo de luz (*bloom*) según su clase espectral, dibuja las estelas orbitales mediante barrido semitransparente y proyecta las flechas de los vectores $\vec{v}_t$ y $\vec{F}_g$ sobre los planetas.

Fase 3: Interfaz HUD Analógica y Flujo de Navegación **(puede ser AI Generated)**

- **Objetivo:** Maquetar en HTML/CSS la interfaz estilo consola espacial e implementar la máquina de estados en HUDController.ts.
- **Entregable verificable:** Flujo completo navegable: pantalla de Mapa Estelar para elegir la estrella → transición al simulador → panel lateral para configurar masa, radio y velocidad tangencial de nuevos cuerpos → botones de pausa, velocidad de simulación y reinicio.

Fase 4: Telemetría Gráfica, Comparador Teórico y Pruebas

- **Objetivo:** Conectar la salida de computeTelemetry() con los paneles de datos del HUD y las barras dinámicas de energía (K, U, E_{total}).
- **Entregable verificable:** Tabla comparativa en vivo que muestra lado a lado el **Valor Teórico vs. Valor Simulado** (*Velocidad Orbital, Fuerza Centrípeta y Período Orbital*) junto con el porcentaje de error, lista para la demostración final en clase.